Deva Dharshini
192472350

SAVEETHA SCHOOL OF ENGINEERING

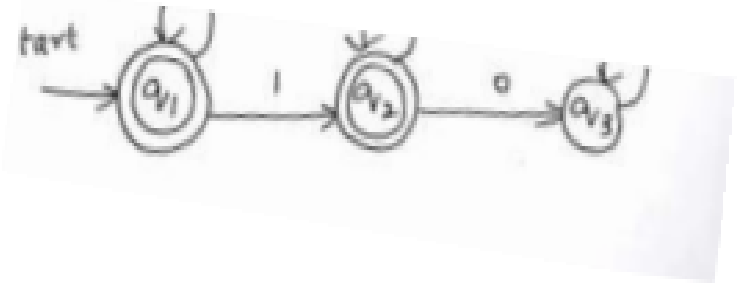
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

COMPUTER SCIENCE AND ENGINEERING

CSA13 – THEORY OF COMPUTATION

ASSIGNMENT – 2

		CO	BTL
1	Write closure properties of regular languages with respect to compliment, kleen star, reversal and substitution	2	4
2	Construct a Non Deterministic Finite automata with ε-transitions to represent the language denoted by the regular expression (a+b)*(ab)*	2	3
3	Give an informal description of the language denoted by the regular expression (0+1)*00	2	3
4	Justify that the language L consisting of all strings with an equal number of a's and b's over the alphabet □□={a,b} is not regular using pumping lemma .	2	4

5	Apply Arden's Theorem and derive the regular expression for the language represented by the given automata 	2	3

Question 1: Write closure properties of regular languages with respect to compliment, kleen star, reversal and substitution

Introduction

A class of languages is closed under an operation if applying that operation to languages of the class always produces another language of the same class. The regular languages are the languages accepted by DFAs, NFAs and ε-NFAs, and the languages described by regular expressions. These three descriptions are equivalent, so to prove a closure property we may use whichever one is most convenient.

Property 1: Closure under complement

Statement: If L is a regular language over an alphabet Σ, then its complement L' = Σ* − L is also regular.

Proof (construction):

1. Since L is regular, there is a DFA M = (Q, Σ, δ, q0, F) with L(M) = L.
2. Make sure M is a complete DFA, meaning every state has a transition on every symbol of Σ. If some transitions are missing, add a dead (trap) state that loops to itself on all symbols and send every missing transition to it.
3. Build M' = (Q, Σ, δ, q0, Q − F). The states, the transitions and the start state are unchanged. Only the final states are swapped: final states become non-final and non-final states become final.
4. For any string w, the machine M reaches exactly the same state as M' because the transitions are identical. So w is accepted by M' exactly when δ*(q0, w) is in Q − F, that is, exactly when w is not accepted by M.
5. Therefore L(M') = Σ* − L. A DFA exists for it, so the complement is regular.

Important remark: This swapping trick works only for a DFA. For an NFA, swapping the final states does not give the complement, because a string may have several paths, and one path ending in a swapped final state does not mean all paths do.

Example: Let L be the set of strings over {0,1} that end in 1. The DFA has two states: A (start, last symbol is not 1) and B (final, last symbol is 1). On 0 both states go to A, and on 1 both go to B. Swapping the final states makes A the only final state. The new DFA accepts the strings over {0,1} that are empty or end in 0, which is exactly the complement of L.

Property 2: Closure under Kleene star

Statement: If L is regular, then L* = { w1 w2 … wk | k ≥ 0 and each wi is in L } is also regular.

Proof using regular expressions: If L is described by a regular expression R, then L* is described by R*. A regular expression for L* exists, so L* is regular.

Proof using an automaton: Let N = (Q, Σ, δ, q0, F) be an NFA for L. Build an ε-NFA N' as follows:

1. Add a new start state qs, and make qs a final state. This ensures the empty string is accepted, since ε is always in L*.
2. Add an ε-transition from qs to the old start state q0.
3. Add an ε-transition from every old final state in F back to q0. This lets the machine start reading another word of L right after finishing one.
4. The final states of N' are qs together with all states of F.

A string is accepted by N' exactly when it can be cut into zero or more pieces, each of which takes N from q0 to a final state. That is exactly the definition of L*.

Important remark: We add a new start state instead of simply making q0 final. If q0 has incoming transitions, making it final would wrongly accept extra strings that are not in L*.

Example: Let L = {ab}. The automaton for L has the path q0 –a→ q1 –b→ q2, with q2 final. After the construction, qs is final, qs –ε→ q0, and q2 –ε→ q0. The new automaton accepts ε, ab, abab, ababab, and so on, which is the language (ab)*.

Property 3: Closure under reversal

Statement: If L is regular, then its reversal L^R = { w^R | w ∈ L } is regular, where w^R is the string w written backwards.

Proof using an automaton: Let N = (Q, Σ, δ, q0, F) be an NFA for L. Build an NFA N^R as follows:

1. Reverse the direction of every transition. If N has p –a→ q, then N^R has q –a→ p.
2. The old start state q0 becomes the only final state of N^R.
3. Add a new start state s with ε-transitions to every state in F. This is needed because N may have several final states.

If w takes N from q0 to a final state, then w^R takes N^R from that final state back to q0. Hence N^R accepts w^R exactly when N accepts w.

Proof using regular expressions: Define the reversal of a regular expression R by structural induction.

- ∅^R = ∅, ε^R = ε, and a^R = a for every symbol a
- (R1 + R2)^R = R1^R + R2^R
- (R1 R2)^R = R2^R R1^R (the order of concatenation is swapped)
- (R*)^R = (R^R)*

Since the reversal of every regular expression is again a regular expression, L^R is regular.

Example: If L is described by ab*c, then L^R is described by c b* a. If L = {ab, abb, abbb, …}, then L^R = {ba, bba, bbba, …}.

Property 4: Closure under substitution

Definition: A substitution σ assigns to every symbol a of Σ a language σ(a) over another alphabet Γ. It is extended to strings by σ(ε) = {ε} and σ(xa) = σ(x) σ(a). It is extended to a language by σ(L) = the union of σ(w) over all w in L. In other words, each symbol of every string is replaced by any string of the language assigned to it. A homomorphism is the special case in which every σ(a) is a single string.

Statement: If L is regular and σ(a) is regular for every symbol a, then σ(L) is regular.

Proof by structural induction on a regular expression R for L, where σ(a) is described by a regular expression Ra:

- σ(∅) = ∅ and σ(ε) = ε
- σ(a) = Ra for every symbol a
- σ(R1 + R2) = σ(R1) + σ(R2)
- σ(R1 R2) = σ(R1) σ(R2)
- σ(R*) = (σ(R))*

In every case the result is again a regular expression, so σ(L) is regular. In practice, the new expression is obtained by replacing each symbol a in R with its assigned expression Ra.

Example: Let L be described by 0*1, and let σ(0) = a + b and σ(1) = cd. Then σ(L) is described by (a + b)* cd. This is regular. For instance, the string 001 in L is replaced by one of aacd, abcd, bacd or bbcd.

Summary table

Operation	**Regular languages closed?**	**Main idea of the proof**
Complement	Yes	Complete DFA, swap final and non-final states
Kleene star	Yes	New start and final state, ε-loops back to the old start
Reversal	Yes	Reverse all edges, swap start and final states
Substitution	Yes	Replace each symbol in the regular expression by its regular expression

Conclusion

Regular languages are closed under complement, Kleene star, reversal and substitution. These properties are useful because they allow us to build new regular languages from known ones and to prove that other languages are not regular.

Question 2: Construct a Non Deterministic Finite automata with ε-transitions to represent the language denoted by the regular expression

(a+b)*(ab)*

Introduction

Thompson's construction converts a regular expression into an ε-NFA. Each basic symbol gets a small automaton with one start state and one final state. These small automata are combined using the operations of union, concatenation and star, with ε-transitions as glue. The expression (a+b)*(ab)* has the structure (a+b)* followed by (ab)*. So we build each part and then join them.

Rules of Thompson's construction

1. For a symbol a, use two states i and f with the transition i –a→ f.
2. For a union R1 + R2, add a new start state with ε-transitions to the starts of R1 and R2, and add a new final state that receives ε-transitions from the finals of R1 and R2.
3. For a concatenation R1 R2, connect the final state of R1 to the start state of R2 with an ε-transition.
4. For a star R*, add a new start state and a new final state. Add ε-transitions from the new start to the start of R, and from the new start to the new final state (for zero repetitions). Add ε-transitions from the final of R back to the start of R (for repetition), and from the final of R to the new final state.

Step 1: Automaton for a and for b

- For a: q1 –a→ q2
- For b: q3 –b→ q4

Step 2: Automaton for (a+b)

Add a new start state q5 and a new end state q6:

- q5 –ε→ q1 and q5 –ε→ q3
- q2 –ε→ q6 and q4 –ε→ q6

Step 3: Automaton for (a+b)*

Add a new start state q0 and a new final state q7:

- q0 –ε→ q5 and q0 –ε→ q7
- q6 –ε→ q5 (to repeat) and q6 –ε→ q7 (to finish)

Step 4: Automaton for ab

- q9 –a→ q10
- q10 –ε→ q11
- q11 –b→ q12

Step 5: Automaton for (ab)*

Add a new start state q8 and a new final state q13:

- q8 –ε→ q9 and q8 –ε→ q13
- q12 –ε→ q9 and q12 –ε→ q13

Step 6: Concatenate the two parts

Connect the end of (a+b)* to the start of (ab)* with q7 –ε→ q8.

Final ε-NFA

- States: q0, q1, q2, …, q13
- Input alphabet: {a, b}
- Start state: q0
- Final state: q13

Transition table

State	a	b	ε
→ q0	∅	∅	{q5, q7}
q1	{q2}	∅	∅
q2	∅	∅	{q6}
q3	∅	{q4}	∅
q4	∅	∅	{q6}
q5	∅	∅	{q1, q3}
q6	∅	∅	{q5, q7}
q7	∅	∅	{q8}
q8	∅	∅	{q9, q13}
q9	{q10}	∅	∅
q10	∅	∅	{q11}
q11	∅	{q12}	∅
q12	∅	∅	{q9, q13}
* q13	∅	∅	∅

Verification with strings

- The empty string: q0 –ε→ q7 –ε→ q8 –ε→ q13. This is accepted.
- The string ab: q0 –ε→ q5 –ε→ q1 –a→ q2 –ε→ q6 –ε→ q5 –ε→ q3 –b→ q4 –ε→ q6 –ε→ q7 –ε→ q8 –ε→ q13. This is accepted.
- The string ba: use the first part for b and then for a, then ε-moves to q13. This is accepted.
- The string abab: either the first part loops four times, or it is read by (ab)* as ab ab through q9, q10, q11, q12, q9, … and ends at q13. This is accepted.

Simplified equivalent ε-NFA (optional)

A smaller ε-NFA for the same expression uses three states:

- p0 is the start state, with self-loops on a and on b, and an ε-transition to p1
- p1 is the final state, with a transition p1 –a→ p2
- p2 has a transition p2 –b→ p1

The part (a+b)* is the loop at p0, and the part (ab)* is the cycle p1 –a→ p2 –b→ p1.

Remark on the language

Since the empty string belongs to (ab)*, the expression (a+b)*(ab)* includes (a+b)* followed by ε. So it accepts every string over {a, b}. The language is therefore the same as that of (a+b)*, which is Σ*. The construction above is still the correct ε-NFA for the given expression.

Conclusion

Using Thompson's construction, the 14-state ε-NFA with start state q0 and final state q13 represents the regular expression (a+b)*(ab)*.

Question 3: Give an informal description of the language denoted by the regular expression (0+1)*00

Introduction

A regular expression describes a language by combining symbols with union (+), concatenation and star (*). To describe a regular expression in words, we read it from left to right and explain what each part contributes.

Reading the expression part by part

- (0+1) means one symbol which is either 0 or 1.
- (0+1)* means any number of such symbols, including none. So it generates every string over {0,1}, including the empty string.
- 00 is a fixed block of two zeros, placed at the end.

Putting these together, the expression generates any string over {0,1} followed by exactly the two symbols 00.

Informal description

The language denoted by (0+1)*00 is the set of all binary strings that end with 00. In other words, the last two symbols of the string must both be 0. Whatever comes before them can be anything.

Properties of the language

- The shortest string is 00, which has length 2.
- Every string in the language has length at least 2.
- The empty string is not in the language, and the strings 0 and 1 are not either.
- The language is infinite, because the prefix can be arbitrarily long.

Examples of strings in the language

00, 000, 100, 0000, 0100, 1100, 10100, 111100, 1010100

Examples of strings not in the language

ε, 0, 1, 01, 10, 11, 001, 010, 101, 1001, 1010, 1101

Each of these fails because the last two symbols are not both 0.

Supporting automaton (DFA)

A DFA with three states recognises this language:

- s0 is the start state. Here the last symbol is 1, or nothing has been read.
- s1 means exactly one 0 has just been read at the end.
- s2 means at least two 0s have just been read at the end. This is the final state.

State	0	1
→ s0	s1	s0

State	0	1
s1	s2	s0
* s2	s2	s0

For the string 10100: s0 –1→ s0 –0→ s1 –1→ s0 –0→ s1 –0→ s2. It ends in the final state, so it is accepted.

For the string 1001: s0 –1→ s0 –0→ s1 –0→ s2 –1→ s0. It ends in a non-final state, so it is rejected.

Conclusion

The regular expression (0+1)*00 denotes the language of all strings over {0,1} that end with 00.

Question 4: Justify that the language L consisting of all strings with an equal number of a's and b's over the alphabet □□={a,b} is not regular using pumping lemma

.**Introduction**

The pumping lemma is a necessary condition that every regular language must satisfy. We use it to prove that a language is not regular by contradiction: assume the language is regular, and then show that the pumping condition fails.

Statement of the pumping lemma

If L is a regular language, then there is a constant $p \geq 1$ (the pumping length) such that every string w in L with $|w| \geq p$ can be split into three parts w = xyz satisfying:

1. $|y| \geq 1$
2. $|xy| \leq p$
3. For every $i \geq 0$, the string x y^i z is in L

Proof that L is not regular

Step 1: Assume, for contradiction, that L is regular. Then it has a pumping length p.

Step 2: Choose a string. Take w = a^p b^p, which consists of p copies of a followed by p copies of b. This string has p a's and p b's, so it is in L. Its length is 2p, which is at least p.

Step 3: Consider any split w = xyz that satisfies conditions 1 and 2.

- Since $|xy| \leq p$, both x and y lie entirely inside the first p symbols of w, which are all a's.
- So y consists only of a's, say y = a^k, and $k \geq 1$ because $|y| \geq 1$.

Step 4: Pump the string. Take i = 2. Then $x y^2 z = a^{(p+k)} b^p$.

- The number of a's is p + k.
- The number of b's is still p.
- Since $k \geq 1$, we have $p + k \neq p$.

So $x y^2 z$ has more a's than b's and is not in L.

Step 5: Contradiction. The pumping lemma says x y^i z must be in L for every $i \geq 0$, but for i = 2 it is not. This contradicts condition 3.

Step 6: Conclusion. Our assumption was wrong, so L is not regular. ■

Checking with another value of i

Taking i = 0 also gives a contradiction. Then $x z = a^{(p-k)} b^p$, which has fewer a's than b's, so it is not in L.

Why the choice of string matters

If we had chosen a string like (ab)^p instead, the pumping lemma would not give a contradiction, because y could be ab and every pumped string would still have equal a's and b's. The string a^p b^p

works because the condition $|xy| \leq p$ forces y to contain only a's.

Alternative proof using closure properties

Regular languages are closed under intersection. The regular language a*b* intersected with L gives { a^n b^n | $n \geq 0$ }. This language is known to be non-regular. If L were regular, its intersection with a*b* would also be regular, which is a contradiction. Hence L is not regular.

Intuitive reason

A finite automaton has only a fixed finite number of states. To compare the count of a's and b's, it would have to remember an unbounded count, which needs unlimited memory. A finite automaton does not have that.

Conclusion

Using the pumping lemma with the string a^p b^p, we showed that the language of all strings with an equal number of a's and b's is not regular.

Question 5: Apply Arden's Theorem and derive the regular expression for the language represented by the given automata

Introduction

Arden's theorem is used to convert a finite automaton into an equivalent regular expression. We write one equation for each state, describing the strings that lead to that state, and solve the equations using the theorem.

Statement of Arden's theorem

Let P and Q be regular expressions over Σ, and suppose P does not contain ε. Then the equation

R = Q + R P

has a unique solution, given by

R = Q P*

Proof of the theorem

Substitute R = Q + RP into itself repeatedly:

- R = Q + RP
- R = Q + (Q + RP)P = Q + QP + RP²
- R = Q + QP + QP² + … + QP^n + RP^(n+1)

So R contains Q P^i for every i. Because P does not contain ε, the term R P^(n+1) contributes only strings longer than n+1 symbols beyond Q, so any string in R belongs to some Q P^i, which means R = Q P*. Conversely, Q P* satisfies the equation. ■

Rules for writing the equations

1. Write an equation for every state. The right-hand side collects every incoming transition as (source state) · (symbol).
2. For the initial state, add ε to its equation.
3. Solve the equations by substitution, applying Arden's theorem to remove self-loops.
4. The final regular expression is the union (+) of the expressions of all final states.

Reading the given automaton

The automaton has three states: q1, q2 and q3.

- q1 is the start state and a final state.
- q2 is a final state. There is a transition q1 –1→ q2.
- q3 is a non-final state. There is a transition q2 –0→ q3.
- Each state has a self-loop. The loop labels are not clearly visible in the image, so I used the standard reading: q1 loops on 0, q2 loops on 1, and q3 loops on both 0 and 1.

Transitions

State	On 0	On 1
→ * q1	q1	q2
* q2	q3	q2
q3	q3	q3

Step 1: Write the equations

- $q1 = \varepsilon + q1 \cdot 0$
- $q2 = q1 \cdot 1 + q2 \cdot 1$
- $q3 = q2 \cdot 0 + q3 \cdot 0 + q3 \cdot 1$

The ε in the first equation is there because q1 is the initial state. The state q1 is entered only by its own loop on 0.

Step 2: Solve for q1

$q1 = \varepsilon + q1 \cdot 0$

This matches the form R = Q + RP with Q = ε and P = 0. Since P = 0 does not contain ε, Arden's theorem applies:

$q1 = \varepsilon \cdot 0^* = 0^*$

Step 3: Solve for q2

$q2 = q1 \cdot 1 + q2 \cdot 1$

Substitute q1 = 0*:

$q2 = 0^*1 + q2 \cdot 1$

This matches R = Q + RP with Q = 0*1 and P = 1. By Arden's theorem:

$q2 = 0^*1 \cdot 1^* = 0^*11^*$

Step 4: Solve for q3 (for completeness)

$q3 = q2 \cdot 0 + q3 \cdot (0+1)$

Substitute q2 = 0*11*:

$q3 = 0^*11^*0 + q3 \cdot (0+1)$

By Arden's theorem:

$q3 = 0^*11^*0(0+1)^*$

The state q3 is not final, so it does not contribute to the answer.

Step 5: Combine the final states

The final states are q1 and q2, so the regular expression of the automaton is

$R = q1 + q2 = 0^* + 0^*11^*$

Step 6: Simplify

Factor out 0*:

R = 0*(ε + 11*)

Since ε + 11* = 1*, we get

R = 0*1*

Answer

The regular expression for the language represented by the given automaton is 0*1*.

Verification

- The empty string, 0, 00, 1, 11, 011 and 00111 are accepted. They end in q1 or q2.
- The string 10 reaches q3 through q1 –1→ q2 –0→ q3 and is rejected. Every string in which a 1 is followed later by a 0 is rejected.
- This agrees with 0*1*, whose strings have all the 0s before all the 1s.

Conclusion

Applying Arden's theorem to the equations of the three states gives q1 = 0*, q2 = 0*11* and q3 = 0*11*0(0+1)*. Combining the final states q1 and q2 gives the regular expression 0*1*.

If your diagram has different loop labels, tell me and I will redo Question 5 with them. Only the equations in Step 1 change.